HW Checker: Design and Implementation Plan

Oct 20, 2026 · @no name

1. Overview, goals and non-goals

The system is a LangGraph-based agent that, twice a week, checks the homework of about 40 second-year students in a .NET 9 Web API course, reports what is present and what is missing, and proposes fixes as a PR to each student's repo after manual approval. A first version must be in use at the start of next week; the system will keep evolving while in use and will later be handed over to another developer.

Course context. Students build a single project incrementally over the semester; each assignment adds a few dozen lines of code. Most work in fixed pairs, but each has their own repo. AI use is forbidden, and the expected code level is relatively basic. At some point the projects use SQL Server, first through a direct connection (ADO.NET) and later through EF Core.

Goals

- Check each repo against the general requirements and the requirements of every assignment up to the current one (regression).
- Build and actually run the project and verify HTTP behavior, not only read the code.
- Find only real, substantive problems: precision before recall.
- Produce a Hebrew report for the reviewer, a grades file per assignment, and plain-English feedback for the student.
- Propose minimal, student-level fixes, and publish them as a PR only after manual approval.
- Collect AI-use signals and present them as a flag for human review.
- Be easy to extend (a new assignment = a data file) and easy to hand over to another developer.

Non-goals (for now)

- Automatic disqualification on suspected AI use.
- A final grade without human oversight.
- Multiple users, permissions, and server-side automatic scheduling.
- Support for languages or frameworks other than .NET 9 Web API.

2. Requirements

The table lists the functional requirements and their priority for the first version; "Should" = included if time allows, otherwise done manually.

ID	Requirement	Priority
F1	Ingest the Excel file (name, partner's name, GitHub link) and merge duplicate rows into pairs	Must
F2	Clone or update each repo; read the commit hash and commit history	Must
F3	Build and run the project in a container against a shared SQL Server	Must
F4	Check general requirements and the requirements of all assignments 1..N	Must
F5	HTTP checks against expected status codes, including allowed alternatives	Must
F6	Filter findings through judge rails (deterministic + LLM)	Must
F7	Hebrew report per repo, grades file per assignment, plain-English feedback	Must
F8	UI to start a round, track progress, review and approve	Must
F9	Publish a PR to the student's repo only after approval	Should
F10	Targeted fixes verified by a rebuild	Should
F11	Use the partner's findings as a hint for a re-check	Must
F12	AI-use signals (commit history, techniques not yet taught)	Next phase

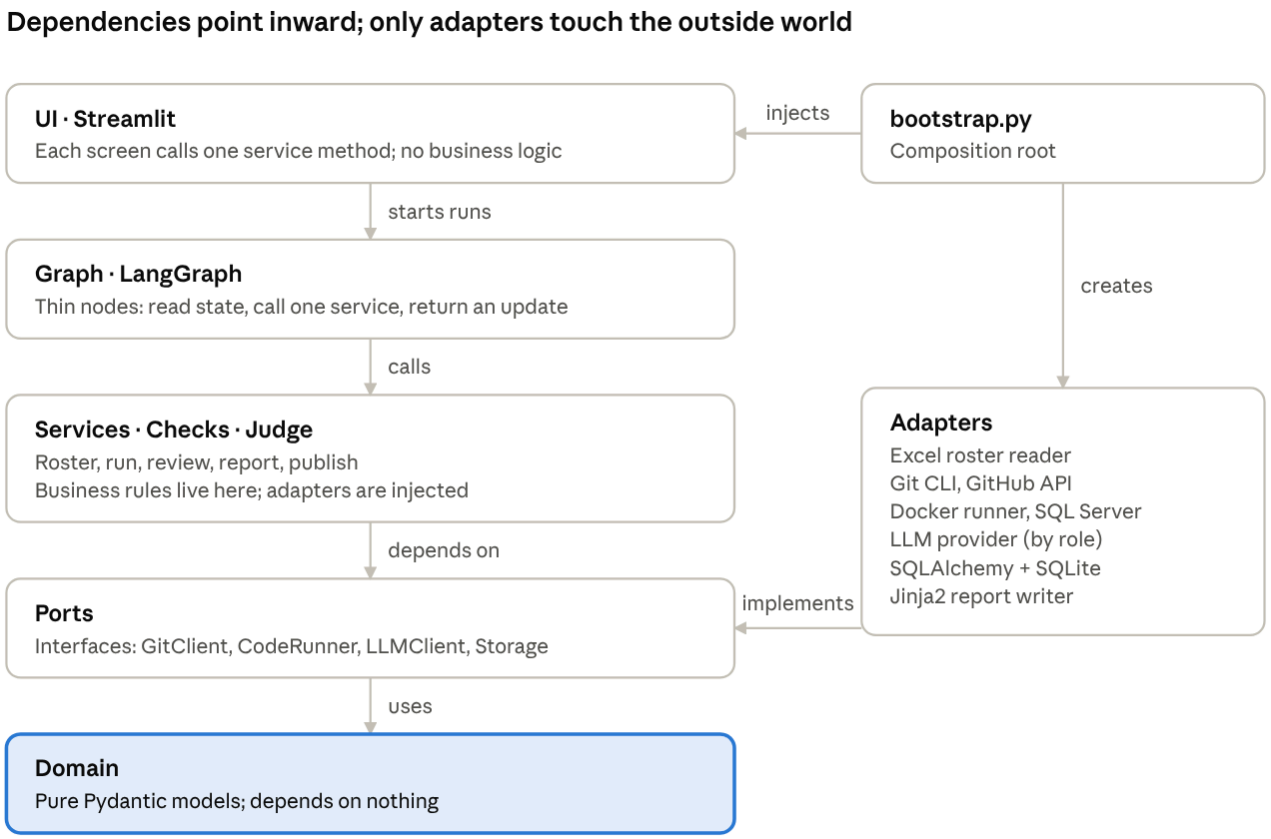
Non-functional requirements

- **Maintainability:** layered architecture, SOLID, single responsibility per component, clean code, documented decisions (ADRs).
- **Extensibility:** a new assignment is added as data files, without changing core code.
- **Scalability:** bounded parallelism (3–4 repos at a time on a local machine); move to a server and a central database without a rewrite.

- **Reliability:** a failure in one repo does not stop the round; a stopped round resumes from where it left off.
- **Precision:** every finding carries evidence (file, line, quote) and a reference to a requirement.
- **Safety:** student code runs only in an isolated container; secrets live only in `.env`.
- **Cost:** a cheap model for simple checks, a strong model for the judge and fixes; model choice in configuration.
- **Environment:** Windows, Docker Desktop, .NET 9 SDK, Python 3.12.

3. Layered architecture and dependency rules

The system uses a layered Ports & Adapters (hexagonal) architecture, so every external component (Excel, Git, GitHub, Docker, LLM, SQLite, UI) can be replaced without touching the logic. Dependencies point inward only, toward the domain.



layered architecture · dependencies point inward

The arrows show dependency direction: each layer on the left depends only on the layer below it, the adapters implement the ports, and only `bootstrap.py` knows both sides and wires them together.

Layer	Responsibility	May depend on
domain	Pure models and business rules (repo, pair, requirement, finding)	Nothing
ports	Interfaces (Protocol/ABC) for every external capability	domain
services	The logic: ingestion, checking, judging, reporting, publishing	domain, ports
checks, judge	Checks and judge; registered in a registry	domain, ports
graph	LangGraph: state, thin nodes, builders	services, domain
adapters	Concrete implementations of the ports	ports, domain, external libraries
bootstrap	Composition root: creates adapters and injects them	Everything
ui	Streamlit; calls services only, through bootstrap	services, domain

Mandatory rules

- A graph node is thin: it receives state, calls one service, and returns a state update. It holds no business logic.
- One service = one responsibility (SRP). A service never creates adapters itself; they are injected (DIP).
- Only `bootstrap.py` imports concrete adapters.
- The UI knows neither LangGraph nor the adapters, so it can later be replaced by FastAPI + React.
- Adding a check, an assignment or an LLM provider is done by addition (registry/configuration), not by changing existing code (OCP).
- Every LLM output passes through a Pydantic model; no free text crosses layers.

4. Project structure

The folder structure mirrors the layers one-to-one, and assignment requirements live outside the code as data.

```
hw-checker/
├── CLAUDE.md                    # working rules for the implementing agent
├── README.md
├── pyproject.toml
├── .env.example
├── docs/
│   ├── design.md                # export of this document
│   └── adr/                     # 0001-ports-and-adapters.md ...
├── assignments/                 # requirements as data, not code
│   ├── _general/requirements.yaml
│   └── hw01/
│       ├── requirements.yaml
│       ├── checks.py            # optional: custom checks (plugin)
│       └── reference/           # optional: reference code
├── docker/
│   ├── dotnet-runner.Dockerfile
│   └── sqlserver.compose.yml
├── src/hw_checker/
│   ├── domain/
│   ├── ports/
│   ├── adapters/
│   │   ├── roster_excel/
│   │   ├── git_local/
│   │   ├── github_api/
│   │   ├── docker_runner/
│   │   ├── llm/
│   │   └── storage_sqlite/
│   ├── services/
│   ├── checks/
│   │   ├── deterministic/
│   │   ├── llm_review/
│   │   └── registry.py
│   ├── judge/
│   ├── graph/
│   │   ├── state.py
│   │   ├── nodes/
│   │   └── builders.py
│   ├── reporting/templates/
│   ├── config/
│   ├── bootstrap.py
│   └── ui/
├── workspace/                   # clones and artifacts (in .gitignore)
```

```
└── tests/
    ├── unit/
    ├── integration/
    └── fixtures/                # sample repo, sample Excel
```

Tooling: Python 3.12, uv, ruff, mypy (strict), pytest, pre-commit, Pydantic v2, pydantic-settings, SQLAlchemy 2, LangGraph, Docker SDK for Python, PyGithub, openpyxl, Jinja2, Streamlit. Exact versions are chosen and pinned in stage 0.

5. Domain model

The unit of work is a repo, not a student; a pair is a group of two separate repos whose findings serve as hints for each other. All models are Pydantic v2, immutable where possible, with no external dependencies.

Model	Key fields	Notes
Student	name, github_url	Identified by normalized URL
Pair	pair_id, members (1–2 Students)	A student working alone = a pair of one
RepoTarget	repo_id, owner, name, url, student, pair_id	
RosterWarning	kind, message, row	Missing partner, invalid URL, asymmetric duplicate
Assignment	id (hw01), order, title, requirements, taught_techniques	Loaded from YAML
Requirement	id, title, description, severity, kind, allowed_alternatives, http_probes	kind: structure / code / http / build
HttpProbe	method, path, body, expected_status, allowed_statuses, notes	
Severity	critical / major / minor	minor does not lower the grade in v1
Evidence	file_path, line_start, line_end, quote	Required for every code finding
Finding	id, requirement_id, assignment_id, severity, message_he, message_en, evidence, source (deterministic/llm), confidence	

Model	Key fields	Notes
JudgedFinding	finding, verdict (accepted/rejected), reason	Rejected findings are kept for calibration
ExecutionResult	build_status, run_status, failure_kind, logs_excerpt, probe_results	failure_kind separates student issues from environment issues
RepoRunResult	repo, commit_sha, results per assignment, judged_findings, score, status	
FixProposal	repo, branch_name, patches, diff, verified	
ReviewDecision	repo_run_id, approved/rejected/edited, reviewer_notes	
Run	run_id, assignment_id, started_at, status, repo_runs	One round

Run failure classification (failure_kind): `build_failed`, `startup_failed`, `db_unreachable`, `hardcoded_connection_unresolved`, `timeout`, `environment_error`. Only `build_failed` and `startup_failed` (when the environment is healthy) count as student issues; the rest are flagged for manual review.

6. Ports and adapters

Every external capability is defined as a narrow interface in `ports/` and implemented in `adapters/`; in tests, every adapter is replaced by an in-memory fake.

Port	Responsibility	v1 adapter	Future replacement
RosterReader	Read the student list	ExcelRosterReader (openpyxl)	Google Sheets, CSV
GitClient	clone/fetch, commit sha, log, branch, commit, push	LocalGitClient (git CLI via subprocess)	
CodeHost	Open PRs, check permissions	GitHubCodeHost (PyGithub)	GitLab

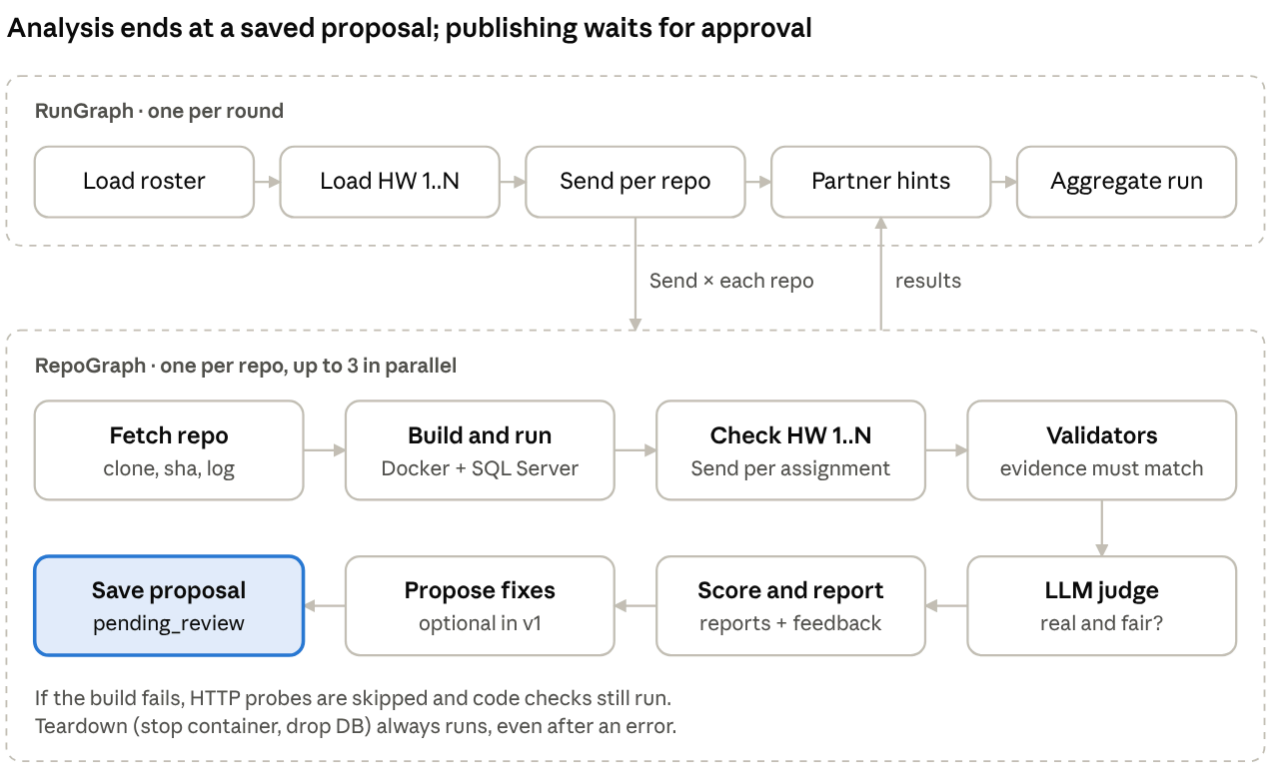
Port	Responsibility	v1 adapter	Future replacement
CodeRunner	build, run, health check, HTTP probes, cleanup	DockerCodeRunner	Remote runner
DatabaseProvisioner	Create and drop a database per repo	SqlServerProvisioner	
LLMClient	Call with structured output, by role name	LangChainLLMClient (init_chat_model)	Any provider
Storage (repositories)	RunRepository, FindingRepository, ProposalRepository	SQLAlchemy + SQLite	PostgreSQL
AssignmentSource	Load assignments and requirements	YamlAssignmentSource	Database / editing UI
ReportWriter	Write reports and the grades file	Jinja2 to Markdown/CSV files	
Clock	Current time	SystemClock	FixedClock in tests

Principles

- Narrow interfaces (ISP): GitClient knows nothing about PRs; CodeHost knows nothing about cloning.
- LLMClient receives a role (for example `reviewer`, `judge`, `fixer`), not a model name; the role → model mapping lives in configuration.
- Every adapter documents the exceptions it raises, mapped to domain exceptions (for example `RepoUnavailableError`).

7. LangGraph design

The graph has three levels: a round graph that fans repos out in parallel, a subgraph per repo, and an inner fan-out per assignment 1..N for regression. Analysis ends at a saved proposal; publishing is a separate service outside the graph.



graph flow · round, repo and assignments

The round sends each repo to its own subgraph (up to 3 in parallel); inside it, each assignment 1..N is checked separately, and analysis stops at a saved proposal awaiting approval before publishing.

Level 1: round graph (RunGraph)

1. `load_roster` – read the Excel file, merge into pairs, collect warnings.
2. `load_assignments` – load `_general` and assignments 1..N.
3. `fan_out_repos` – Send per repo (parallelism capped in configuration, default 3).
4. `share_partner_hints` – after both partners finish, one partner's findings are passed as a hint for a re-check of the other (see note).
5. `aggregate_run` – collect results, produce the grades file, set round status.

Level 2: repo subgraph (RepoGraph)

1. `fetch_repo` – clone/fetch, commit sha, commit history.
2. `build_and_run` – build, provision the database, run, health check. If the build fails, skip HTTP checks (conditional edge) and continue to code checks.
3. `fan_out_assignments` – Send per assignment 1..N (level 3).
4. `validate_findings` – deterministic validators.
5. `judge_findings` – LLM judge.
6. `score_and_report` – score, Hebrew report, English feedback.

7. `propose_fixes` (should) – fixes, rebuild, diff.
8. `persist_proposal` – save the proposal with status `pending_review`.
9. `teardown` – stop the container and drop the database (always, even on failure).

Level 3: assignment check (AssignmentCheck) – for each assignment, run the checks registered for it: deterministic (structure, build, HTTP) and an LLM review for each code requirement. Results merge into state through a list reducer.

State

- `RunState`: run_id, assignment_id, repos, warnings, repo_results (reducer: append).
- `RepoState`: repo, commit_sha, execution, findings (reducer: append), judged, report, proposal, partner_hints, errors.
- State holds only ids and small models; logs and large files live on disk or in the database and are referenced by path.

Persistence and errors

- SQLite checkpointer, one thread per round, so a stopped round resumes from its last point.
- Every node is wrapped in error handling: an error is recorded in the repo's `errors` and the graph continues to `teardown`. A failed repo does not stop the round.
- RetryPolicy for nodes with network calls (LLM, GitHub), with a timeout.

Note on partner hints: in v1, hints are simple: after both repos finish, an accepted finding in one that was not found in the other triggers one focused check in the other. The finding is never copied automatically.

8. Requirements engine and YAML schema

A new assignment is added as a folder under `assignments/` with no code change: the loader validates the YAML against a Pydantic model, and the registry auto-discovers custom checks in `checks.py`. Checking assignment N loads `_general` plus every assignment whose order is ⩽ N.

Example schema (illustrative; finalized in stage 3)

```
id: hw03
order: 3
title: "Products controller with GET/POST"
taught_techniques:        # what has been taught so far; also used for AI
signals
  - controllers
  - routing attributes
```

```
  - IActionResult
requirements:
  - id: hw03-r1
    kind: structure
    title: "ProductsController exists under Controllers/"
    severity: major
    check: file_exists
    params: { glob: "**/Controllers/ProductsController.cs" }
  - id: hw03-r2
    kind: code
    title: "GET by id returns NotFound when product is missing"
    severity: major
    description: >
      The action should not return 200 with null.
    allowed_alternatives:
      - "Returning 204 NoContent is acceptable"
  - id: hw03-r3
    kind: http
    title: "GET /api/products returns 200"
    severity: critical
    probe:
      method: GET
      path: /api/products
      expected_status: 200
      allowed_statuses: [200]
lenient_notes:
  - "Do not require async/await at this stage"
  - "Naming in Hebrew transliteration is acceptable"
```

Requirement kinds and checks

kind	Performed by	Example
structure	Deterministic check (file_exists, folder_layout, project_references)	A Controllers folder exists
build	Deterministic, from the build result	The project builds without errors
http	Deterministic, HTTP probe against the running app	POST returns 201 or 200
code	LLM reviewer, evidence required	Null check before returning
custom	A function in `checks.py` registered with a decorator	Anything that does not fit the template

Rules

- `lenient_notes` and `allowed_alternatives` go into the reviewer's and judge's prompts, and are also used by the validators.
- General requirements (`_general`): basic clean code, folder structure, reasonable use of HTTP status codes. They are phrased leniently, to fit second-year students.
- Changing an old assignment's requirement affects regression in later rounds, so every file is kept in Git and its hash is stored with the round's results.

9. Running code in Docker with SQL Server

Each repo is built and run in an isolated .NET 9 container, against one shared SQL Server container with a separate database per repo. This is the highest-risk stage, so it starts with an isolated spike before the full implementation.

Flow per repo

1. Locate the main `.csproj` (the Web project). If there are several, choose by `Microsoft.NET.Sdk.Web`; if still ambiguous, `environment_error` with an explanation.
2. Create a dedicated database (`hw_<repo_id>_<run_id>`) in the shared container.
3. Override the connection string: environment variables in the form `ConnectionStrings__<Name>` for every name found in `appsettings*.json`.
4. If a connection string is hard-coded (ADO.NET), replace it in the local working copy only and record the replacement in the report. If it cannot be identified safely: `hardcoded_connection_unresolved`, with no grade penalty.
5. Create the schema: for EF Core, run migrations if present; for a direct connection, use a SQL script the student included (if any) or a reference script from the assignment folder.
6. `dotnet build`, then `dotnet run` in the container with a dynamically mapped port.
7. Health check: wait until the port responds (timeout in configuration).
8. Run the HTTP probes from all assignments 1..N, in the defined order.
9. Teardown: stop the container and drop the database, always.

Decisions

- One shared SQL Server is started once per round (compose), because a SQL Server container per repo is too heavy for a local machine.
- The student's container has no external network access other than the shared database, and has memory and CPU limits.
- Build and run logs are saved to a file; only a relevant excerpt goes into the report.

- Working-copy changes made to run the project (connection string) never go into the fixes branch.

Questions the spike must answer: average build time, memory use when running in parallel, Docker Desktop behavior on Windows with volumes, and whether a single SDK image is enough.

10. LLM layer and judge rails

The LLM is used only where a deterministic check is not enough, and every finding it produces passes two filtering layers before it reaches a report. Guiding principle: missing a minor note is better than making a wrong one.

Roles and models

Role	Use	Model tier
reviewer	Review code against `code` requirements	Cheap to mid
judge	Decide whether a finding is substantive and level-appropriate	Strong
fixer	Write a targeted fix and its explanation	Strong
writer	Phrase plain-English feedback	Cheap

The role → provider + model mapping lives in configuration. The provider will be chosen once API access through Copilot is clarified; the code does not depend on the provider.

Prompt rules

- Every prompt is a versioned template file in the repo, not a string in code.
- The prompt includes the requirement, `allowed_alternatives`, `lenient_notes`, the students' level, and only the relevant files (not the whole repo).
- Structured output is mandatory (a Pydantic model); a finding without Evidence is rejected at parse time.
- Low temperature; bounded input size, with long files trimmed by relevance.

Judge rails: two layers

1. **Deterministic validators** (cheap, always run):
   - The file and lines in Evidence exist in the repo.
   - The quote matches the code at those lines (whitespace-normalized comparison).
   - The requirement_id exists and belongs to assignments 1..N.

- The finding does not fall under `allowed_alternatives` (for example, a status listed in `allowed_statuses`).
- No duplicate of the same finding.

2. **LLM judge** (only on what passed layer 1):
   - Is the finding correct according to the quoted code?
   - Is it substantive relative to the requirement, or minor/stylistic?
   - Is it fair for a second-year student and for what has been taught so far?
   - Output: accepted/rejected + reason + adjusted severity (it may only lower severity, never raise it).

Every rejected finding is stored with its rejection reason and shown, collapsed, on the review screen, so requirements and prompts can be calibrated over time.

AI-use signals (next phase)

In v1, commit history is collected and shown raw. The next phase adds signals: a single huge commit, techniques not listed in `taught_techniques`, unusual code volume, and a style gap relative to earlier rounds. The output is a flag with a reason for human review, never a disqualification.

11. Reports, fixes and publishing

Analysis and publishing are two separate phases: the graph ends with a saved proposal in status `pending_review`, and a deterministic publishing service (no LLM) pushes and opens a PR only for approved proposals.

Three outputs, all from Jinja2 templates

Output	Audience	Language	Destination
Repo report	The reviewer	Hebrew	Review screen + Markdown file
Assignment grades file	Reviewer and lecturer	Hebrew	CSV/Markdown in the reviewer's grades repo, a new file per assignment
Student feedback	The student	Plain English	PR description + `FEEDBACK.md` in the branch

The LLM writes only the message text of each finding; structure, headings and score are set by the template and code, so the format is stable.

Score: a simple, transparent formula in configuration (weight by severity; minor does not lower the grade in v1). The final rubric will be set with the lecturer; the formula is isolated in a single service so it is easy to replace.

Fixes (should, in v1)

1. For each accepted, fixable finding, the fixer returns a targeted patch (search-and-replace in a file) and a short plain-English explanation.
2. Patches are applied on a local branch named `hw-review/<assignment>-<date>`, based on the commit that was checked.
3. Rebuild and rerun the checks; a patch that breaks something is dropped and the finding stays as a comment only.
4. One commit per patch, with a message referencing the requirement.
5. Fixes stay at student level: no refactoring, no renames, no techniques not yet taught.

Publishing

- Allowed only for a proposal in status `approved`.
- Push the branch to the student's repo (or to a fork if there is no permission) and open a PR with the feedback as its description.
- Idempotent: publishing again updates the same PR rather than opening a new one.
- Every action is logged with the PR link.
- Until the service is ready: reports and feedback are produced as files and sent manually.

12. User interface and storage

The v1 UI is a thin Streamlit app that calls only the service layer; storage is SQLite behind the repository pattern. Both choices allow a later move to FastAPI + React and to PostgreSQL without touching the core.

Screens

Screen	What happens there
Start a round	Choose an assignment, upload the Excel file, show ingestion warnings, run button
Round progress	Table of repos with status (queued, running, done, failed), duration, and a link to details; auto-refresh

Screen	What happens there
Review	Per repo: accepted findings, rejected findings (collapsed), build/HTTP results, fix diff, report; approve, reject and note buttons
Assignments	View each assignment's requirements as loaded (read-only in v1)

UI principles

- One clear action per screen; error states shown in plain language with a suggested next step.
- Running a round does not block the UI: the graph runs in a background process and the UI reads status from the database.
- No business logic in the UI; every button calls one service method.

Storage

- Tables: runs, repo_runs, findings, judged_findings, proposals, review_decisions, publish_log.
- Each table is accessed only through a dedicated repository behind the `Storage` port.
- Schema migrations through Alembic from the start, so moving to PostgreSQL is a connection-string change.
- The LangGraph checkpointer uses a separate SQLite file, so technical state is not mixed with business data.
- Large artifacts (clones, logs, diffs) go in `workspace/` by run_id; the database stores paths.

13. Configuration, errors, security and testing

Every value that may change lives in configuration, every expected error maps to a domain exception, and every layer is tested on its own with fakes.

Configuration

- `pydantic-settings` with a `.env` file (secrets) and a `config.yaml` file (everything else); `.env.example` documents every variable.
- Examples: parallelism, timeouts, role → model mapping, workspace path, Docker image, score weights, branch name.
- No magic values in code.

Logging and errors

- Structured logging (JSON to file, readable on the console), with run_id and repo_id on every line.
- Domain exception hierarchy: `HwCheckerError` ← `RosterError`, `RepoUnavailableError`, `BuildError`, `ExecutionEnvironmentError` (not `EnvironmentError`, which is a Python built-in name), `LLMError`, `PublishError`.
- Adapters map external library exceptions to domain exceptions; upper layers never catch a general `Exception`, except at a graph node boundary.

Security

- Student code runs only in a container, with no external network and with resource limits.
- The GitHub token and LLM keys live only in `.env`, which is in `.gitignore`; pre-commit includes a secret scan.
- Content from a student's repo (code, comments, README) is data only: prompts mark it as such, so a comment in their code cannot change the checking instructions (prompt injection).
- Publishing is possible only after explicit approval.

Testing strategy

Type	What is tested	How
Unit	domain, services, validators, loader, pair merging	pytest with a fake for every port
Contract	Every adapter satisfies its port	A shared test run against both the fake and the real adapter
Integration	Docker runner, Git, SQLite	Marked `@pytest.mark.integration`, run locally
Graph	Graph flow and error paths	Fake LLM and fake runner
Golden	Checking output on a fixed sample repo	Compared with a saved result; catches prompt regressions

Every implementation stage ends with ruff, mypy and pytest (unit) passing cleanly.

14. Implementation plan by stage

The first phase has 17 implementation stages; 15 of them are required for the start of next week. Each stage runs on its own branch, goes through review, and is merged into `main` only after approval (the full working protocol is in the Planner & Reviewer Agent Guide).

Stage	Topic	Main deliverables	Done when	Priority	Depends on
0	Infrastructure	Folder structure, pyproject, uv, ruff, mypy, pytest, pre-commit, configuration, logging, CLAUDE.md, README, first ADRs	Tools run cleanly on an empty project	Must	–
1	Domain and ports	All models from section 5, all interfaces from section 6, exception hierarchy	Validation tests pass; no implementations	Must	0
2	Excel ingestion and pairs	ExcelRosterReader, RosterService	A sample Excel file yields unique repos and pairs; edge cases covered	Must	1
3	Requirements engine	YAML schema, loader, registry, 1..N selection, YAML for the current assignment	A new assignment folder is picked up without code changes	Must	1
4	Git	LocalGitClient	Clone and re-run both work; sha and history are read	Must	1
5a	Run spike	Isolated experiment script: build+run a sample repo against SQL Server	The sample repo answers HTTP; spike answers recorded in an ADR	Must	0
6	Storage	SQLAlchemy models, Alembic, repositories	A full round is saved and loaded, with tests	Must	1
5b	Full runner	DockerCodeRunner, SqlServerProvisioner, connection string override, probes, teardown	Every failure kind is classified correctly in integration tests	Must	5a, 1
7	LLM layer	LLMClient, role mapping, structured output, retry	A real call returns a valid model; switching models is config-only	Must (blocked: provider)	1
8	Checks	Deterministic checks + LLM reviewer + prompts as files	The sample repo yields structured findings with evidence	Must	3, 5b, 7
9	Judge rails	Validators + LLM judge + storing rejected findings	Fake findings are filtered out in tests	Must	8
10	Graph assembly	RunGraph, RepoGraph, Send, checkpointer, error isolation, partner hints	A round over 2–3 repos runs end to end and is saved	Must	2–9

Stage	Topic	Main deliverables	Done when	Priority	Depends on
11	Reports	Jinja2 templates, score, grades file, English feedback	All three outputs produced from a real round	Must	10
12	Streamlit UI	The four screens from section 12, background run	A full round runs from the UI without a terminal	Must	11
13	PR publishing	PublishService, GitHubCodeHost	A PR opens only for an approved proposal; re-publishing updates it	Should	6, 11
14	Fixes	fixer, patch application, rebuild, diff in the UI	A fix that breaks the build is dropped automatically	Should	10
15	First real run	A round over real repos, calibrating requirements and prompts, bug fixes	A full round completes and results are checked manually	Must	12

Practical order: 0 → 1 → 5a (early, because of the risk) → 2, 3, 4 → 6 → 5b → 7 → 8 → 9 → 10 → 11 → 12 → 13, 14 if time allows → 15. Stages 2, 3 and 4 are independent of each other and can be done in any order.

Definition of done for every stage

- [ ] The deliverables in the table exist, and nothing more (no creep into other stages).
- [ ] ruff, mypy and pytest (unit) pass cleanly.
- [ ] New tests cover the new code, including failure paths.
- [ ] README, `.env.example` and ADRs updated as needed.
- [ ] The implementer's report was received, reviewed and approved.

15. Risks, open questions and roadmap

The highest risk is running the projects in Docker on Windows against SQL Server; the only immediate blocker is choosing the LLM provider.

Risks

Risk	Impact	Mitigation
Docker/SQL Server on Windows is slow or unstable	No HTTP checks	Early spike (5a); fall back to code-only checks, flagged

Risk	Impact	Mitigation
Connection strings hard-coded in varied forms	Repos do not run	Detect and replace in the local copy; classify as `hardcoded_connection_unresolved` with no penalty
The LLM is too strict or invents findings	Wrong feedback to students	Required evidence, validators, judge, allowed_alternatives, manual approval
API access through Copilot unavailable	Stage 7 blocked	LLMClient abstraction; choose another provider in configuration
No write access to student repos	No PR	PR from a fork, or send feedback manually
Short timeline	Partial version	Must/should priorities; stages 13–14 can be deferred
Handover to another developer	Knowledge loss	ADRs, CLAUDE.md, this document, golden tests

Open questions

- [ ] Does the Copilot account allow API access from Python code, and to which models?
- [ ] Grading rubric (with the lecturer).
- [ ] Point in time to check: repo state at run time, or the last commit before the deadline (with the lecturer).
- [ ] Collaborator access to student repos.
- [ ] The exact requirement list for the current assignment (for stage 3).
- [ ] A sample repo or reference project at the current stage (for stages 5a and 8).
- [ ] Whether the projects already have Swagger.
- [ ] The project's final name.

Roadmap after the first version

1. AI-use signals (commit history, techniques not yet taught, style gap).
2. Caching by commit sha + requirement hash, so regression on an unchanged repo is nearly free.
3. Automatic scheduling (Windows Task Scheduler, later a server).
4. Selective review: only repos flagged as problematic go to manual approval.
5. FastAPI on top of the service layer, later React and multiple users.
6. PostgreSQL instead of SQLite.
7. Editing requirements from the UI.

8. Checking students' unit tests once the course reaches that stage.